SECTION 1

Training and Visualization

- **Decision trees:** Handle classification, regression, and multi-output tasks with little preprocessing—no feature scaling required.
- **`DecisionTreeClassifier`:** Fitted on features such as petal length and width; `max_depth` limits how deep the tree grows for clearer diagrams.
- **`export_graphviz`:** Writes a `.dot` description; Graphviz converts it to PNG or PDF for inspection of nodes, splits, and leaves.
- **Node annotations:** Sample counts, per-class value vectors, and Gini impurity at each node summarize how training data flows through the tree.
- **White-box models:** Trees yield interpretable rules, unlike many ensemble or neural models treated as black boxes.

SECTION 2

Making Predictions

- **Root-to-leaf traversal:** Each internal node asks a threshold question on one feature; the path ends at a leaf class or value.
- **Class probabilities:** The leaf's training class fractions become `predict_proba` outputs; `predict` chooses the argmax class.
- **Decision boundaries:** Splits appear as axis-aligned lines; depth limits how finely the plane is partitioned.
- **Binary CART trees:** Scikit-Learn builds two child nodes per split (yes/no), though other algorithms allow multiway branches.

SECTION 3

The CART Training Algorithm

- **CART:** Recursively splits data using one feature and threshold that minimize weighted impurity in the two children.
- **Greedy growth:** Splits are chosen locally at each node; globally optimal trees are computationally intractable for large data.
- **Stopping rules:** Growth halts at `max_depth` or when splits no longer reduce impurity; `min_samples_split` and related hyperparameters add further stops.
- **Prediction cost:** Balanced trees need about O(log m) node visits per instance, largely independent of feature count.
- **Training cost:** All features may be scanned at every node, giving roughly O(n × m log m) complexity; `presort` can help only on small sets.

SECTION 4

Gini Impurity and Entropy

- **Gini impurity:** Measures how often a random label draw from a node would be misclassified; pure leaves have Gini zero.
- **Entropy:** Shannon-style information measure; zero when only one class remains in a node.
- **`criterion`:** Defaults to `gini`; `entropy` is optional and usually yields similar trees with slightly more balanced splits.
- **Practical choice:** Gini is marginally faster; either criterion is acceptable for most classification trees.

SECTION 5

Regularization Hyperparameters

- **Nonparametric flexibility:** Unrestricted trees memorize training noise; regularization constrains structure to improve generalization.
- **`max_depth`:** Primary knob to shallow the tree and reduce overfitting.
- **`min_samples_split` / `min_samples_leaf`:** Require enough instances before splitting or in leaves, smoothing decision regions.
- **`max_leaf_nodes` and `max_features`:** Cap tree size or the features considered per split.
- **Pre-pruning vs post-pruning:** Scikit-Learn trees use split constraints; other packages may prune splits that fail significance tests.

SECTION 6

Regression Trees

- **`DecisionTreeRegressor`:** Leaves predict the mean target of training samples reaching that leaf.
- **Split criterion:** CART minimizes MSE between targets and leaf means instead of Gini or entropy.
- **Piecewise constant surfaces:** Predictions form flat regions; deeper trees create finer stair-steps.
- **Overfitting:** Unregularized regression trees track noise; `min_samples_leaf` and depth limits yield smoother fits.

SECTION 7

Instability and Limitations

- **Orthogonal boundaries:** Splits are perpendicular to axes, so rotated linearly separable data may get overly jagged borders.
- **High sensitivity:** Removing or perturbing a few points—or stochastic feature subsampling—can change the entire tree.
- **`random_state`:** Fixes randomness in training for reproducible forests of single trees.
- **Remedy preview:** Random forests average many trees to stabilize predictions, as developed in the next chapter.